

Desarrollo para calcular el tenure hasta la primera alerta

Arquitectura, estrategia de volumen y código Spark SQL + Hive

Documento técnico de implementación

Objetivo: Para cada party_id que registró una primera alerta válida, obtener la fecha efectiva mínima en person_restricted, calcular el tiempo transcurrido hasta la primera alerta y analizar en qué meses o rangos de tenure se concentra el volumen de alertas.

Elemento	Definición adoptada
Inicio de relación	MIN(data_date) por party_id en person_restricted.
Evento objetivo	Primera fecha válida de creación de alerta por party_id.
Tenure	Diferencia entre first_alert_date y relationship_start_date.
Regla de alerta	Un party_id no puede colocar otra alerta durante los siguientes 7 años.
Tecnología	Spark SQL con Hive Metastore y tablas Parquet.
Estrategia de volumen	Filtrar temprano, seleccionar solo party_id y data_date, materializar resultados compactos y validar primero con una muestra determinística.

Fecha de preparación: agosto de 2026

Contenido

1. Alcance y definición del tenure.
2. Arquitectura de datos y tablas resultantes.
3. Supuestos y limitaciones.
4. Configuración de Spark SQL + Hive.
5. Construcción de eventos únicos de alerta.
6. Validación de la regla de siete años.
7. Primera alerta válida y cohorte del estudio.
8. Muestra determinística para pruebas.
9. Cálculo de MIN(data_date) en person_restricted.
10. Cálculo del tenure en días, meses y años.
11. Validaciones de calidad.
12. Distribuciones por mes y bandas de tenure.
13. Estabilidad por año.
14. Estrategia para volúmenes extremadamente grandes.
15. Criterios para decidir cómo usar el tenure.
16. Checklist de ejecución y entregables.

Importante: Este documento calcula y describe el tenure entre personas que registraron una alerta. La concentración de alertas en un rango no equivale a una mayor probabilidad. Para estimar riesgo se necesita posteriormente el denominador: personas activas o elegibles en cada rango de tenure.

1. Alcance y definición del tenure

El análisis inicial responde la siguiente pregunta:

Pregunta analítica: Entre las personas que registraron una primera alerta, ¿cuánto tiempo transcurrió desde su primera fecha efectiva registrada hasta la creación de la alerta, y en qué rangos se concentra el mayor volumen?

1.1 Fórmula conceptual

```
relationship_start_date = MIN(person_restricted.data_date) por party_id
first_alert_date        = MIN(fecha válida de creación de alerta) por party_id

tenure_days_at_alert    = first_alert_date - relationship_start_date
tenure_months_at_alert  = meses completos entre ambas fechas
```

1.2 Qué sí y qué no concluye este análisis

Sí permite	No permite todavía
Describir el tiempo hasta la primera alerta entre quienes alertaron.	Afirmar que un rango tiene mayor probabilidad de alerta.
Identificar picos de volumen por mes o banda de tenure.	Comparar riesgo sin conocer cuántas personas estaban expuestas en cada tenure.
Evaluar estabilidad del patrón entre años.	Concluir causalidad o vulnerabilidad por tenure.
Definir hipótesis y ventanas para análisis posteriores.	Usar el tenure como única señal de riesgo.

2. Arquitectura de datos

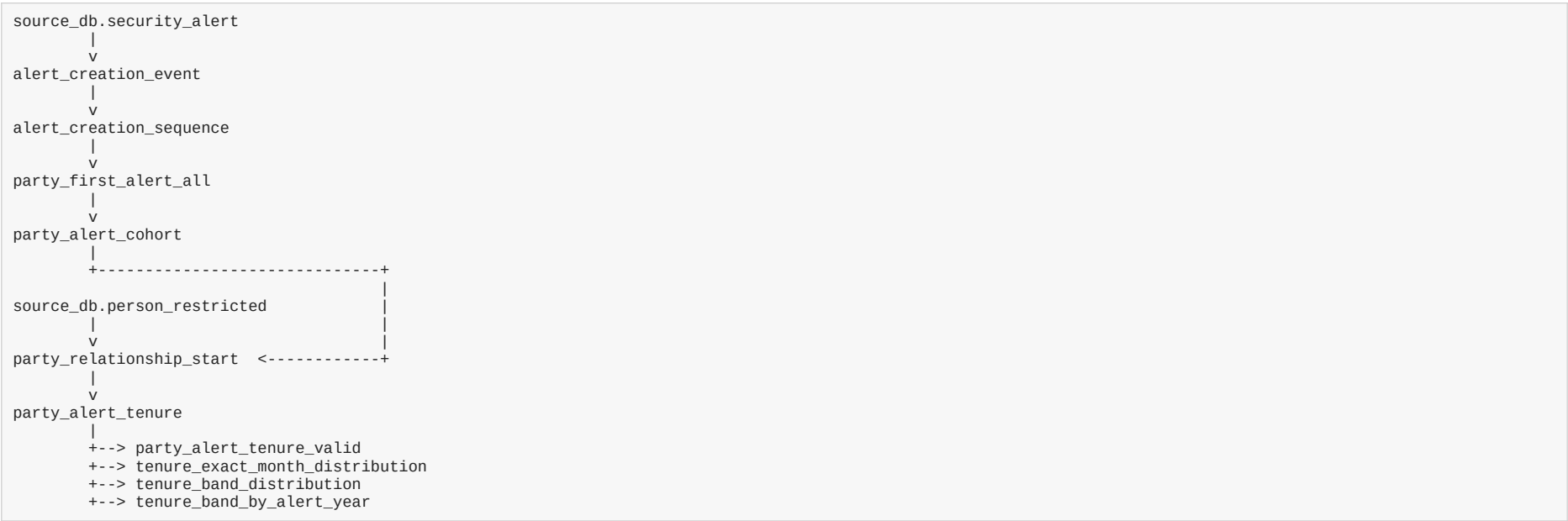


Tabla resultante	Granularidad	Propósito
alert_creation_event	party_id + alert_create_date	Eliminar snapshots repetidos de una misma creación.
alert_creation_sequence	Evento de creación	Detectar múltiples fechas y validar la regla de 7 años.
party_first_alert_all	Una fila por party_id	Conservar la primera alerta observada.

Tabla resultante	Granularidad	Propósito
party_alert_cohort	Una fila por party_id	Limitar el análisis al período de estudio.
party_relationship_start	Una fila por party_id	Conservar MIN(data_date) de person_restricted.
party_alert_tenure	Una fila por party_id	Calcular tenure y estatus de calidad.
tenure_*_distribution	Agregado	Analizar picos, bandas y estabilidad temporal.

3. Supuestos y limitaciones

- person_restricted.data_date se considera una fecha efectiva y su mínimo por party_id representa el inicio real de relación registrado.
- La tabla person_restricted puede contener múltiples filas para un mismo party_id debido a diferentes productos; estas filas no se cuentan como tenure adicional.
- Un party_id no puede colocar una nueva alerta durante los 84 meses posteriores a la creación de una alerta.
- La tabla security_alert puede repetir la misma alerta durante años porque mantiene snapshots de una alerta vigente.
- La primera versión analiza alertas entre 2022-01-01 y 2025-12-31; las fechas son parámetros y deben ajustarse.
- Los tipos de alerta válidos deben ser confirmados por el dueño del dato.
- El cálculo inicial no requiere nombre, apellido, SSN, dirección ni otra PII.

Control de confidencialidad: En toda la construcción se seleccionan únicamente identificadores técnicos y fechas necesarias. No se materializan nombres, SSN ni datos de contacto.

4. Paso 0 - Configuración de Spark SQL + Hive

Se habilita Adaptive Query Execution para permitir ajustes dinámicos de particiones y manejo de ciertos problemas de skew. El número de shuffle partitions no se fija todavía; debe calibrarse con métricas reales del cluster.

```
SET spark.sql.adaptive.enabled = true;
SET spark.sql.adaptive.coalescePartitions.enabled = true;
SET spark.sql.adaptive.skewJoin.enabled = true;
SET spark.sql.sources.partitionOverwriteMode = dynamic;

CREATE DATABASE IF NOT EXISTS fraud_lab;
```

4.1 Parámetros del estudio

```
DROP TABLE IF EXISTS fraud_lab.cfg_tenure_project;

CREATE TABLE fraud_lab.cfg_tenure_project
USING PARQUET
AS
```

```
SELECT
  CAST('2022-01-01' AS DATE) AS study_start_date,
  CAST('2025-12-31' AS DATE) AS study_end_date,
  84 AS alert_duration_months,
  100 AS sample_modulus;
```

Interpretación de sample_modulus = 100: la muestra determinística seleccionará aproximadamente 1 de cada 100 party_id.

5. Paso 1 - Configurar tipos de alerta válidos

La configuración siguiente es provisional. Debe ajustarse después de confirmar cuál tipo representa la colocación de una nueva alerta por la persona.

```
DROP TABLE IF EXISTS fraud_lab.cfg_valid_alert_type;

CREATE TABLE fraud_lab.cfg_valid_alert_type
USING PARQUET
AS
SELECT *
FROM VALUES
  (1, 'ACTIVE_DUTY_ALERT', 0),
  (2, 'INITIAL_FRAUD_ALERT', 1),
  (3, 'EXTENDED_FRAUD_ALERT', 1),
  (4, 'CALIFORNIA_ALERT', 0),
  (5, 'LOUISIANA_ALERT', 0),
  (6, 'TEXAS_ALERT', 0)
AS config(
  sec_alert_typ_num,
  alert_type_description,
  use_as_outcome
);
```

```
SELECT *
FROM fraud_lab.cfg_valid_alert_type
ORDER BY sec_alert_typ_num;
```

Gate de negocio: Si Extended Fraud Alert no representa una nueva creación, debe excluirse del outcome antes de continuar.

6. Paso 2 - Perfil básico de la fuente de alertas

Este paso confirma cobertura temporal y volumen. Para exploración se usa approx_count_distinct; los conteos exactos se ejecutan sobre tablas ya reducidas.

```
SELECT
  COUNT(*) AS source_row_count,
  approx_count_distinct(party_id) AS approximate_party_count,
  MIN(CAST(cratt_dte AS DATE)) AS minimum_create_date,
  MAX(CAST(cratt_dte AS DATE)) AS maximum_create_date,
  MIN(CAST(data_date AS DATE)) AS minimum_snapshot_date,
  MAX(CAST(data_date AS DATE)) AS maximum_snapshot_date
FROM source_db.security_alert
WHERE party_id IS NOT NULL
  AND cratt_dte IS NOT NULL;
```

```

SELECT
  CAST(sec_alert_typ_num AS INT) AS sec_alert_typ_num,
  COUNT(*) AS snapshot_row_count,
  approx_count_distinct(party_id) AS approximate_party_count
FROM source_db.security_alert
WHERE party_id IS NOT NULL
  AND crat_dte IS NOT NULL
GROUP BY CAST(sec_alert_typ_num AS INT)
ORDER BY sec_alert_typ_num;

```

7. Paso 3 - Crear eventos únicos de alerta

La tabla security_alert puede tener muchos snapshots de la misma alerta. Se reduce a una fila por party_id y fecha de creación.

```

DROP TABLE IF EXISTS fraud_lab.alert_creation_event;

CREATE TABLE fraud_lab.alert_creation_event
USING PARQUET
AS
SELECT
  s.party_id,
  CAST(s.crat_dte AS DATE) AS alert_create_date,
  sort_array(
    collect_set(CAST(s.sec_alert_typ_num AS INT))
  ) AS observed_alert_type_set,
  MIN(CAST(s.data_date AS DATE)) AS first_snapshot_date,
  MAX(CAST(s.data_date AS DATE)) AS last_snapshot_date,
  MIN(CAST(s.expire_dte AS DATE)) AS minimum_expire_date,
  MAX(CAST(s.expire_dte AS DATE)) AS maximum_expire_date,
  MAX(CAST(s.upd_tsp AS TIMESTAMP)) AS latest_update_timestamp,
  COUNT(*) AS source_snapshot_count
FROM source_db.security_alert AS s
INNER JOIN fraud_lab.cfg_valid_alert_type AS t
  ON CAST(s.sec_alert_typ_num AS INT) = t.sec_alert_typ_num
  AND t.use_as_outcome = 1
WHERE s.party_id IS NOT NULL
  AND s.crat_dte IS NOT NULL
  AND s.data_date IS NOT NULL
GROUP BY
  s.party_id,
  CAST(s.crat_dte AS DATE);

```

7.1 Validar unicidad y reducción

```

SELECT
  party_id,
  alert_create_date,
  COUNT(*) AS row_count
FROM fraud_lab.alert_creation_event
GROUP BY party_id, alert_create_date
HAVING COUNT(*) > 1
LIMIT 100;

```

```

SELECT
  'ALERT_CREATION_EVENTS' AS metric,
  COUNT(*) AS metric_value
FROM fraud_lab.alert_creation_event

UNION ALL

SELECT
  'PARTIES_WITH_ALERT',
  COUNT(DISTINCT party_id)
FROM fraud_lab.alert_creation_event;

```

Resultado esperado: La consulta de unicidad debe retornar cero filas. La cantidad de eventos debe ser considerablemente menor que la cantidad de snapshots.

8. Paso 4 - Validar múltiples fechas de creación y la regla de 7 años

Se ordenan los eventos por persona. Cualquier creación ocurrida antes de 84 meses desde la anterior se marca para revisión y no se cuenta automáticamente como una segunda alerta.

```

DROP TABLE IF EXISTS fraud_lab.alert_creation_sequence;

CREATE TABLE fraud_lab.alert_creation_sequence
USING PARQUET
AS
WITH ordered_events AS
(
  SELECT
    e.*,
    LAG(alert_create_date) OVER
    (
      PARTITION BY party_id
      ORDER BY alert_create_date
    ) AS previous_alert_create_date
  FROM fraud_lab.alert_creation_event AS e
)
SELECT
  *,
  CASE
    WHEN previous_alert_create_date IS NULL
      THEN 'FIRST_OBSERVED_ALERT'
    WHEN alert_create_date < add_months(previous_alert_create_date, 84)
      THEN 'CREATION_WITHIN_7_YEARS'
    ELSE 'CREATION_AFTER_7_YEARS'
  END AS sequence_status,
  CASE
    WHEN previous_alert_create_date IS NULL
      THEN CAST(NULL AS INT)
    ELSE CAST(
      FLOOR(
        months_between(
          alert_create_date,
          previous_alert_create_date
        )
      ) AS INT
    ) AS months_since_previous_alert
END AS months_since_previous_alert

```

```
FROM ordered_events;
```

```
SELECT
    sequence_status,
    COUNT(*) AS event_count,
    approx_count_distinct(party_id) AS approximate_party_count
FROM fraud_lab.alert_creation_sequence
GROUP BY sequence_status
ORDER BY event_count DESC;
```

```
SELECT
    party_id,
    previous_alert_create_date,
    alert_create_date,
    months_since_previous_alert,
    observed_alert_type_set
FROM fraud_lab.alert_creation_sequence
WHERE sequence_status = 'CREATION_WITHIN_7_YEARS'
ORDER BY party_id, alert_create_date
LIMIT 200;
```

9. Paso 5 - Crear la primera alerta válida por persona

Para el análisis de tenure se conserva la fecha más antigua observada por `party_id`. La tabla resultante tiene una fila por persona.

```
DROP TABLE IF EXISTS fraud_lab.party_first_alert_all;

CREATE TABLE fraud_lab.party_first_alert_all
USING PARQUET
AS
WITH ranked_alerts AS
(
    SELECT
        e.*,
        ROW_NUMBER() OVER
        (
            PARTITION BY party_id
            ORDER BY alert_create_date, first_snapshot_date
        ) AS alert_order
    FROM fraud_lab.alert_creation_event AS e
)
SELECT
    party_id,
    alert_create_date AS first_alert_date,
    observed_alert_type_set AS first_alert_type_set,
    first_snapshot_date,
    last_snapshot_date,
    COALESCE(
        maximum_expire_date,
        add_months(alert_create_date, 84)
    ) AS first_alert_expire_date,
    source_snapshot_count
FROM ranked_alerts
WHERE alert_order = 1;
```



```

SELECT
    party_id,
    COUNT(*) AS row_count
FROM fraud_lab.party_first_alert_all
GROUP BY party_id
HAVING COUNT(*) > 1
LIMIT 100;

```

10. Paso 6 - Seleccionar la cohorte del estudio

La cohorte se limita a personas cuya primera alerta observada ocurrió dentro del período de estudio.

```

DROP TABLE IF EXISTS fraud_lab.party_alert_cohort;

CREATE TABLE fraud_lab.party_alert_cohort
USING PARQUET
AS
SELECT
    a.*
FROM fraud_lab.party_first_alert_all AS a
CROSS JOIN fraud_lab.cfg_tenure_project AS c
WHERE a.first_alert_date
      BETWEEN c.study_start_date AND c.study_end_date;

```

```

SELECT
    year(first_alert_date) AS alert_year,
    COUNT(*) AS party_count
FROM fraud_lab.party_alert_cohort
GROUP BY year(first_alert_date)
ORDER BY alert_year;

```

```

SELECT
    first_alert_type_set,
    COUNT(*) AS party_count
FROM fraud_lab.party_alert_cohort
GROUP BY first_alert_type_set
ORDER BY party_count DESC;

```

11. Paso 7 - Crear una muestra determinística

Antes de escanear todo person_restricted se valida la lógica con aproximadamente 1 % de los party_id de la cohorte. La muestra se hace por persona, no por filas.

```

DROP TABLE IF EXISTS fraud_lab.party_alert_cohort_sample;

CREATE TABLE fraud_lab.party_alert_cohort_sample
USING PARQUET
AS
SELECT
    a.*
FROM fraud_lab.party_alert_cohort AS a
WHERE pmod(

```

```

        hash(CAST(a.party_id AS STRING)),
        100
    ) = 0;

```

```

SELECT
    COUNT(*) AS sampled_party_count,
    MIN(first_alert_date) AS minimum_alert_date,
    MAX(first_alert_date) AS maximum_alert_date
FROM fraud_lab.party_alert_cohort_sample;

```

```

SELECT
    year(first_alert_date) AS alert_year,
    COUNT(*) AS sampled_party_count
FROM fraud_lab.party_alert_cohort_sample
GROUP BY year(first_alert_date)
ORDER BY alert_year;

```

12. Paso 8 - Calcular MIN(data_date) en person_restricted para la muestra

Este es el paso de mayor costo. La optimización principal es leer solo party_id y data_date, y aplicar un LEFT SEMI JOIN para conservar únicamente las personas de la cohorte.

```

DROP TABLE IF EXISTS fraud_lab.party_relationship_start_sample;

CREATE TABLE fraud_lab.party_relationship_start_sample
USING PARQUET
AS
SELECT
    p.party_id,
    MIN(CAST(p.data_date AS DATE)) AS relationship_start_date
FROM source_db.person_restricted AS p
LEFT SEMI JOIN fraud_lab.party_alert_cohort_sample AS a
    ON p.party_id = a.party_id
WHERE p.party_id IS NOT NULL
    AND p.data_date IS NOT NULL
    AND CAST(p.data_date AS DATE) <= CAST('2025-12-31' AS DATE)
GROUP BY p.party_id;

```

```

SELECT
    COUNT(*) AS sampled_alert_parties,
    SUM(CASE WHEN s.party_id IS NOT NULL THEN 1 ELSE 0 END)
        AS parties_found_in_person_restricted,
    SUM(CASE WHEN s.party_id IS NULL THEN 1 ELSE 0 END)
        AS parties_not_found
FROM fraud_lab.party_alert_cohort_sample AS a
LEFT JOIN fraud_lab.party_relationship_start_sample AS s
    ON a.party_id = s.party_id;

```

13. Paso 9 - Calcular y validar el tenure en la muestra

Se calculan días, meses completos y estatus de calidad. El cálculo de meses completos evita contar como mes completo un período donde el día de la alerta aún no alcanza el día de inicio.

```
DROP TABLE IF EXISTS fraud_lab.party_alert_tenure_sample;

CREATE TABLE fraud_lab.party_alert_tenure_sample
USING PARQUET
AS
SELECT
  a.party_id,
  s.relationship_start_date,
  a.first_alert_date,
  a.first_alert_type_set,
  CASE
    WHEN s.relationship_start_date IS NULL
      THEN CAST(NULL AS INT)
    ELSE datediff(
      a.first_alert_date,
      s.relationship_start_date
    )
  END AS tenure_days_at_alert,
  CASE
    WHEN s.relationship_start_date IS NULL
      THEN CAST(NULL AS INT)
    WHEN s.relationship_start_date > a.first_alert_date
      THEN CAST(NULL AS INT)
    ELSE
      (
        year(a.first_alert_date) * 12
        + month(a.first_alert_date)
      )
      -
      (
        year(s.relationship_start_date) * 12
        + month(s.relationship_start_date)
      )
      - CASE
          WHEN dayofmonth(a.first_alert_date)
            < dayofmonth(s.relationship_start_date)
            THEN 1
          ELSE 0
        END
  END AS tenure_full_months_at_alert,
  CASE
    WHEN s.relationship_start_date IS NULL
      THEN 'MISSING_RELATIONSHIP_START'
    WHEN s.relationship_start_date > a.first_alert_date
      THEN 'START_AFTER_ALERT'
    WHEN s.relationship_start_date = a.first_alert_date
      THEN 'SAME_DAY_START_AND_ALERT'
    ELSE 'VALID'
  END AS tenure_quality_status
FROM fraud_lab.party_alert_cohort_sample AS a
LEFT JOIN fraud_lab.party_relationship_start_sample AS s
  ON a.party_id = s.party_id;
```

```
SELECT
  tenure_quality_status,
  COUNT(*) AS party_count
```

```
FROM fraud_lab.party_alert_tenure_sample
GROUP BY tenure_quality_status
ORDER BY party_count DESC;
```

```
SELECT
  COUNT(*) AS valid_party_count,
  MIN(tenure_full_months_at_alert) AS minimum_tenure_months,
  MAX(tenure_full_months_at_alert) AS maximum_tenure_months,
  AVG(tenure_full_months_at_alert) AS average_tenure_months,
  percentile_approx(
    tenure_full_months_at_alert,
    array(0.10, 0.25, 0.50, 0.75, 0.90, 0.95),
    10000
  ) AS tenure_percentiles
FROM fraud_lab.party_alert_tenure_sample
WHERE tenure_quality_status IN ('VALID', 'SAME_DAY_START_AND_ALERT');
```

Gate antes del full run: No ejecutar el cálculo completo hasta validar cobertura, tiempos, fechas posteriores a la alerta, distribución plausible y representación de la muestra por año.

14. Paso 10 - Revisar la organización física de person_restricted

```
DESCRIBE FORMATTED source_db.person_restricted;
```

```
SHOW PARTITIONS source_db.person_restricted;
```

SHOW PARTITIONS solo aplica cuando la tabla está particionada. En el plan debe revisarse si Spark está usando PartitionFilters y PushedFilters.

```
EXPLAIN FORMATTED
SELECT
  p.party_id,
  MIN(CAST(p.data_date AS DATE)) AS relationship_start_date
FROM source_db.person_restricted AS p
LEFT SEMI JOIN fraud_lab.party_alert_cohort AS a
  ON p.party_id = a.party_id
WHERE p.party_id IS NOT NULL
  AND p.data_date IS NOT NULL
  AND CAST(p.data_date AS DATE) <= CAST('2025-12-31' AS DATE)
GROUP BY p.party_id;
```

Elemento del plan	Qué revisar
FileScan	Que solo lea las columnas party_id y data_date.
PartitionFilters	Que elimine particiones no necesarias cuando la tabla esté particionada.
PushedFilters	Que los filtros se envíen al lector de Parquet/ORC.

Elemento del plan	Qué revisar
Join	BroadcastHashJoin solo si la cohorte es realmente pequeña; de lo contrario SortMergeJoin.
Exchange	Volumen de shuffle y cantidad de particiones.
HashAggregate	Agregación parcial y final del MIN(data_date).

15. Paso 11 - Ejecución completa en una sola pasada

Cuando la muestra sea satisfactoria, se materializa una fila por party_id de la cohorte. Esta tabla compacta debe reutilizarse y no recalcularse para cada análisis.

```
DROP TABLE IF EXISTS fraud_lab.party_relationship_start;

CREATE TABLE fraud_lab.party_relationship_start
USING PARQUET
AS
SELECT
    p.party_id,
    MIN(CAST(p.data_date AS DATE)) AS relationship_start_date
FROM source_db.person_restricted AS p
LEFT SEMI JOIN fraud_lab.party_alert_cohort AS a
    ON p.party_id = a.party_id
WHERE p.party_id IS NOT NULL
    AND p.data_date IS NOT NULL
    AND CAST(p.data_date AS DATE) <= CAST('2025-12-31' AS DATE)
GROUP BY p.party_id;
```

```
SELECT
    party_id,
    COUNT(*) AS row_count
FROM fraud_lab.party_relationship_start
GROUP BY party_id
HAVING COUNT(*) > 1
LIMIT 100;
```

```
SELECT
    COUNT(*) AS alert_cohort_size,
    SUM(CASE WHEN s.party_id IS NOT NULL THEN 1 ELSE 0 END)
    AS found_party_count,
    SUM(CASE WHEN s.party_id IS NULL THEN 1 ELSE 0 END)
    AS missing_party_count,
    ROUND(
        100.0
        * SUM(CASE WHEN s.party_id IS NOT NULL THEN 1 ELSE 0 END)
        / COUNT(*),
        2
    ) AS coverage_percentage
FROM fraud_lab.party_alert_cohort AS a
LEFT JOIN fraud_lab.party_relationship_start AS s
    ON a.party_id = s.party_id;
```

16. Paso 12 - Alternativa por períodos para cargas extremadamente grandes

Esta estrategia solo es conveniente cuando `person_restricted` está físicamente particionada por año o fecha y el plan confirma `partition pruning`. Si cada ejecución vuelve a leer todos los archivos, será más costosa que una sola pasada.

```
DROP TABLE IF EXISTS fraud_lab.party_relationship_start_by_year;

CREATE TABLE fraud_lab.party_relationship_start_by_year
(
    party_id BIGINT,
    minimum_data_date_in_year DATE
)
USING PARQUET
PARTITIONED BY (source_year INT);
```

```
INSERT OVERWRITE TABLE fraud_lab.party_relationship_start_by_year
PARTITION (source_year = 2000)
SELECT
    p.party_id,
    MIN(CAST(p.data_date AS DATE)) AS minimum_data_date_in_year
FROM source_db.person_restricted AS p
LEFT SEMI JOIN fraud_lab.party_alert_cohort AS a
    ON p.party_id = a.party_id
WHERE CAST(p.data_date AS DATE) >= CAST('2000-01-01' AS DATE)
    AND CAST(p.data_date AS DATE) < CAST('2001-01-01' AS DATE)
    AND p.party_id IS NOT NULL
    AND p.data_date IS NOT NULL
GROUP BY p.party_id;
```

```
DROP TABLE IF EXISTS fraud_lab.party_relationship_start;

CREATE TABLE fraud_lab.party_relationship_start
USING PARQUET
AS
SELECT
    party_id,
    MIN(minimum_data_date_in_year) AS relationship_start_date
FROM fraud_lab.party_relationship_start_by_year
GROUP BY party_id;
```

17. Paso 13 - Calcular el tenure completo

```
DROP TABLE IF EXISTS fraud_lab.party_alert_tenure;

CREATE TABLE fraud_lab.party_alert_tenure
USING PARQUET
AS
SELECT
    a.party_id,
    s.relationship_start_date,
    a.first_alert_date,
    a.first_alert_type_set,
    year(a.first_alert_date) AS first_alert_year,
    month(a.first_alert_date) AS first_alert_calendar_month,
```

```

CASE
  WHEN s.relationship_start_date IS NULL
    THEN CAST(NULL AS INT)
  ELSE datediff(
    a.first_alert_date,
    s.relationship_start_date
  )
END AS tenure_days_at_alert,
CASE
  WHEN s.relationship_start_date IS NULL
    THEN CAST(NULL AS INT)
  WHEN s.relationship_start_date > a.first_alert_date
    THEN CAST(NULL AS INT)
  ELSE
    (
      year(a.first_alert_date) * 12
      + month(a.first_alert_date)
    )
    -
    (
      year(s.relationship_start_date) * 12
      + month(s.relationship_start_date)
    )
    - CASE
        WHEN dayofmonth(a.first_alert_date)
          < dayofmonth(s.relationship_start_date)
          THEN 1
          ELSE 0
        END
  END
END AS tenure_full_months_at_alert,
CASE
  WHEN s.relationship_start_date IS NULL
    THEN CAST(NULL AS DOUBLE)
  WHEN s.relationship_start_date > a.first_alert_date
    THEN CAST(NULL AS DOUBLE)
  ELSE ROUND(
    datediff(
      a.first_alert_date,
      s.relationship_start_date
    ) / 365.25,
    2
  )
END AS tenure_years_at_alert,
CASE
  WHEN s.relationship_start_date IS NULL
    THEN 'MISSING_RELATIONSHIP_START'
  WHEN s.relationship_start_date > a.first_alert_date
    THEN 'START_AFTER_ALERT'
  WHEN s.relationship_start_date = a.first_alert_date
    THEN 'SAME_DAY_START_AND_ALERT'
  ELSE 'VALID'
END AS tenure_quality_status
FROM fraud_lab.party_alert_cohort AS a
LEFT JOIN fraud_lab.party_relationship_start AS s
  ON a.party_id = s.party_id;

```

18. Paso 14 - Validaciones de calidad del tenure

```
SELECT
  tenure_quality_status,
  COUNT(*) AS party_count,
  ROUND(
    100.0 * COUNT(*) / SUM(COUNT(*)) OVER (),
    2
  ) AS percentage
FROM fraud_lab.party_alert_tenure
GROUP BY tenure_quality_status
ORDER BY party_count DESC;
```

```
SELECT
  party_id,
  relationship_start_date,
  first_alert_date
FROM fraud_lab.party_alert_tenure
WHERE tenure_quality_status = 'START_AFTER_ALERT'
ORDER BY first_alert_date, party_id
LIMIT 200;
```

```
SELECT
  party_id,
  relationship_start_date,
  first_alert_date,
  tenure_full_months_at_alert,
  tenure_years_at_alert
FROM fraud_lab.party_alert_tenure
WHERE tenure_full_months_at_alert > 720
ORDER BY tenure_full_months_at_alert DESC
LIMIT 200;
```

El umbral de 720 meses (60 años) es solo un control inicial de plausibilidad; debe ajustarse con el contexto del negocio y la antigüedad real de las fuentes.

19. Paso 15 - Crear la población analítica válida

```
DROP TABLE IF EXISTS fraud_lab.party_alert_tenure_valid;

CREATE TABLE fraud_lab.party_alert_tenure_valid
USING PARQUET
AS
SELECT *
FROM fraud_lab.party_alert_tenure
WHERE tenure_quality_status IN (
  'VALID',
  'SAME_DAY_START_AND_ALERT'
);
```

```
SELECT
  COUNT(*) AS valid_party_count,
  COUNT(DISTINCT party_id) AS distinct_valid_party_count
```



```
FROM fraud_lab.party_alert_tenure_valid;
```

Resultado esperado: Los dos conteos deben ser iguales: una fila analítica válida por party_id.

20. Paso 16 - Estadísticas generales

```
SELECT
  COUNT(*) AS party_count,
  MIN(tenure_full_months_at_alert) AS minimum_months,
  MAX(tenure_full_months_at_alert) AS maximum_months,
  AVG(tenure_full_months_at_alert) AS average_months,
  percentile_approx(
    tenure_full_months_at_alert,
    array(
      0.05,
      0.10,
      0.25,
      0.50,
      0.75,
      0.90,
      0.95,
      0.99
    ),
    10000
  ) AS tenure_percentiles
FROM fraud_lab.party_alert_tenure_valid;
```

La mediana y los percentiles son más robustos que el promedio cuando existe una cola larga de clientes con muchos años de relación.

21. Paso 17 - Distribución por mes exacto de tenure

Esta tabla identifica picos específicos, por ejemplo 3, 5 o 6 meses. Debe revisarse junto con bandas más amplias para evitar sobreinterpretar ruido mensual.

```
DROP TABLE IF EXISTS fraud_lab.tenure_exact_month_distribution;

CREATE TABLE fraud_lab.tenure_exact_month_distribution
USING PARQUET
AS
WITH monthly_counts AS
(
  SELECT
    tenure_full_months_at_alert AS tenure_month,
    COUNT(*) AS alert_party_count
  FROM fraud_lab.party_alert_tenure_valid
  GROUP BY tenure_full_months_at_alert
),
distribution AS
(
  SELECT
    tenure_month,
    alert_party_count,
    SUM(alert_party_count) OVER () AS total_alert_parties
```

```

    FROM monthly_counts
)
SELECT
    tenure_month,
    alert_party_count,
    ROUND(
        100.0 * alert_party_count / total_alert_parties,
        4
    ) AS alert_party_percentage,
    ROUND(
        100.0
        * SUM(alert_party_count) OVER
        (
            ORDER BY tenure_month
            ROWS BETWEEN UNBOUNDED PRECEDING
                AND CURRENT ROW
        )
        / total_alert_parties,
        4
    ) AS cumulative_percentage
FROM distribution;

```

```

SELECT
    tenure_month,
    alert_party_count,
    alert_party_percentage,
    cumulative_percentage
FROM fraud_lab.tenure_exact_month_distribution
ORDER BY alert_party_count DESC
LIMIT 20;

```

22. Paso 18 - Distribución por bandas de tenure

```

DROP TABLE IF EXISTS fraud_lab.tenure_band_distribution;

CREATE TABLE fraud_lab.tenure_band_distribution
USING PARQUET
AS
WITH tenure_bands AS
(
    SELECT
        party_id,
        tenure_full_months_at_alert,
        CASE
            WHEN tenure_full_months_at_alert = 0
            THEN '00_SAME_MONTH'
            WHEN tenure_full_months_at_alert BETWEEN 1 AND 2
            THEN '01_02_MONTHS'
            WHEN tenure_full_months_at_alert BETWEEN 3 AND 5
            THEN '03_05_MONTHS'
            WHEN tenure_full_months_at_alert BETWEEN 6 AND 11
            THEN '06_11_MONTHS'
            WHEN tenure_full_months_at_alert BETWEEN 12 AND 23
            THEN '12_23_MONTHS'
            WHEN tenure_full_months_at_alert BETWEEN 24 AND 35
            THEN '24_35_MONTHS'
            WHEN tenure_full_months_at_alert BETWEEN 36 AND 59
            THEN '36_59_MONTHS'

```

```

        WHEN tenure_full_months_at_alert BETWEEN 60 AND 83
        THEN '60_83_MONTHS'
        ELSE '84_PLUS_MONTHS'
    END AS tenure_band,
CASE
    WHEN tenure_full_months_at_alert = 0 THEN 0
    WHEN tenure_full_months_at_alert BETWEEN 1 AND 2 THEN 1
    WHEN tenure_full_months_at_alert BETWEEN 3 AND 5 THEN 2
    WHEN tenure_full_months_at_alert BETWEEN 6 AND 11 THEN 3
    WHEN tenure_full_months_at_alert BETWEEN 12 AND 23 THEN 4
    WHEN tenure_full_months_at_alert BETWEEN 24 AND 35 THEN 5
    WHEN tenure_full_months_at_alert BETWEEN 36 AND 59 THEN 6
    WHEN tenure_full_months_at_alert BETWEEN 60 AND 83 THEN 7
    ELSE 8
END AS tenure_band_order
FROM fraud_lab.party_alert_tenure_valid
),
band_counts AS
(
    SELECT
        tenure_band,
        tenure_band_order,
        COUNT(*) AS alert_party_count
    FROM tenure_bands
    GROUP BY tenure_band, tenure_band_order
)
SELECT
    tenure_band,
    tenure_band_order,
    alert_party_count,
    ROUND(
        100.0 * alert_party_count
        / SUM(alert_party_count) OVER (),
        4
    ) AS alert_party_percentage
FROM band_counts;

```

```

SELECT
    tenure_band,
    alert_party_count,
    alert_party_percentage
FROM fraud_lab.tenure_band_distribution
ORDER BY tenure_band_order;

```

23. Paso 19 - Validar estabilidad por año

Un pico global puede ser causado por un solo año, una migración de datos o un cambio operacional. El patrón debe compararse entre años de creación de alerta.

```

DROP TABLE IF EXISTS fraud_lab.tenure_band_by_alert_year;

CREATE TABLE fraud_lab.tenure_band_by_alert_year
USING PARQUET
AS
WITH tenure_bands AS
(
    SELECT
        party_id,
        first_alert_year,

```

```

CASE
  WHEN tenure_full_months_at_alert = 0
    THEN '00_SAME_MONTH'
  WHEN tenure_full_months_at_alert BETWEEN 1 AND 2
    THEN '01_02_MONTHS'
  WHEN tenure_full_months_at_alert BETWEEN 3 AND 5
    THEN '03_05_MONTHS'
  WHEN tenure_full_months_at_alert BETWEEN 6 AND 11
    THEN '06_11_MONTHS'
  WHEN tenure_full_months_at_alert BETWEEN 12 AND 23
    THEN '12_23_MONTHS'
  WHEN tenure_full_months_at_alert BETWEEN 24 AND 35
    THEN '24_35_MONTHS'
  WHEN tenure_full_months_at_alert BETWEEN 36 AND 59
    THEN '36_59_MONTHS'
  WHEN tenure_full_months_at_alert BETWEEN 60 AND 83
    THEN '60_83_MONTHS'
  ELSE '84_PLUS_MONTHS'
END AS tenure_band
FROM fraud_lab.party_alert_tenure_valid
),
year_counts AS
(
  SELECT
    first_alert_year,
    tenure_band,
    COUNT(*) AS alert_party_count
  FROM tenure_bands
  GROUP BY first_alert_year, tenure_band
)
SELECT
  first_alert_year,
  tenure_band,
  alert_party_count,
  ROUND(
    100.0 * alert_party_count
    / SUM(alert_party_count)
    OVER (PARTITION BY first_alert_year),
    4
  ) AS percentage_within_alert_year
FROM year_counts;

```

```

SELECT *
FROM fraud_lab.tenure_band_by_alert_year
ORDER BY first_alert_year, alert_party_count DESC;

```

24. Paso 20 - Estadísticas físicas y mantenimiento

```

ANALYZE TABLE fraud_lab.party_first_alert_all
COMPUTE STATISTICS NOSCAN;

ANALYZE TABLE fraud_lab.party_alert_cohort
COMPUTE STATISTICS NOSCAN;

ANALYZE TABLE fraud_lab.party_relationship_start
COMPUTE STATISTICS NOSCAN;

ANALYZE TABLE fraud_lab.party_alert_tenure_valid

```

```
COMPUTE STATISTICS NOSCAN;
```

- No forzar BROADCAST sobre party_alert_cohort sin validar su tamaño real.
- No particionar por party_id: crearía demasiadas particiones pequeñas.
- Mantener person_restricted fuera de consultas repetitivas; leerla una vez para crear party_relationship_start.
- Conservar las tablas compactas en Parquet y materializar cada etapa costosa.
- Usar approx_count_distinct en exploración y conteos exactos únicamente sobre tablas reducidas.
- Revisar skew mediante distribución de filas por party_id antes de aplicar técnicas de salting.

25. Cómo decidir si y cuándo utilizar el tenure

El desarrollo anterior permite conocer la distribución del tenure entre personas que colocaron una alerta. Para decidir cómo usarlo en fases posteriores deben evaluarse varios criterios.

Uso potencial	Cuándo tiene sentido	Riesgo o cautela
Segmentación	Cuando los patrones cambian entre clientes nuevos, tempranos y establecidos.	No convertir una banda en un perfil fijo de vulnerabilidad.
Matching o control	Para comparar alertados y no alertados con antigüedad similar.	Evitar confundir antigüedad con riesgo.
Feature de modelo	Cuando mejora validación temporal y aporta señal adicional.	La relación puede ser no lineal; usar bandas o splines.
Definir historial mínimo	Para separar clientes sin suficiente historia de quienes permiten baselines de 6 o 12 meses.	Puede requerir modelos o reglas distintas para clientes nuevos.
Definir horizonte del label	Cuando los picos son estables y muestran tiempo útil para intervenir.	El volumen absoluto no sustituye la tasa de alerta.

Siguiente paso estadístico: Calcular para cada banda de tenure el número de personas elegibles y la cantidad de nuevas alertas. La tasa alertas / personas expuestas es la medida necesaria para determinar si existe mayor riesgo relativo.

26. Checklist de ejecución

Orden	Acción	Gate de validación
1	Confirmar tipos de alerta válidos.	Definición firmada por el data owner.
2	Crear alert_creation_event.	Una fila por party_id + alert_create_date.

Orden	Acción	Gate de validación
3	Validar regla de 84 meses.	Anomalías revisadas y documentadas.
4	Crear party_first_alert_all y cohorte.	Una fila por party_id; cobertura anual coherente.
5	Ejecutar muestra determinística.	Representación por año y tiempo aceptable.
6	Calcular MIN(data_date) en muestra.	Cobertura alta y pocos START_AFTER_ALERT.
7	Revisar EXPLAIN FORMATTED.	Column pruning, filtros y join adecuados.
8	Ejecutar full run y materializar.	Una fila por party_id en relationship_start.
9	Calcular tenure y estados de calidad.	Población válida y percentiles plausibles.
10	Analizar meses, bandas y años.	Patrones consistentes, no explicados por un solo año.
11	Incorporar denominador en fase posterior.	Tasas por tenure antes de decisiones predictivas.

27. Tablas finales y propósito

Tabla	Uso principal
fraud_lab.alert_creation_event	Eventos únicos de creación de alerta.
fraud_lab.alert_creation_sequence	Control de múltiples creaciones y regla de 7 años.
fraud_lab.party_first_alert_all	Primera alerta observada por party_id.
fraud_lab.party_alert_cohort	Cohorte de primeras alertas dentro del estudio.
fraud_lab.party_relationship_start	Fecha efectiva mínima por party_id.
fraud_lab.party_alert_tenure	Tenure completo y controles de calidad.
fraud_lab.party_alert_tenure_valid	Población analítica depurada.
fraud_lab.tenure_exact_month_distribution	Picos por mes exacto.
fraud_lab.tenure_band_distribution	Concentración por rangos.
fraud_lab.tenure_band_by_alert_year	Estabilidad del patrón por año.

Cierre y siguientes pasos

Conclusión: La estrategia recomendada evita procesar repetidamente la tabla ancha `person_restricted`. Primero reduce las alertas a una cohorte de una fila por persona; después lee solamente `party_id` y `data_date` para materializar `MIN(data_date)`. El resultado permite describir el tiempo hasta la primera alerta y decidir qué rangos de tenure merecen un análisis posterior de tasas y señal predictiva.

- Ejecutar primero la muestra determinística y revisar cobertura, fechas inválidas y duración del job.
- Materializar `party_relationship_start` una sola vez y reutilizarla en análisis posteriores.
- En una fase posterior, incorporar personas sin alerta para calcular tasas por tenure y evaluar riesgo relativo.